SFMC BÉNIN

Plateforme Microservices

Rapport Technique de Projet

Master 1 IFRI · Groupe 3

Architecture Microservices · Next.js · Kong · RabbitMQ · Prisma

Année académique 2025–2026

1. Contexte du projet

SFMC Bénin est une entreprise industrielle spécialisée dans la production et la distribution de matériaux de construction : ciment, fer, briques, granulats. Leurs processus étaient partiellement manuels, avec une faible traçabilité et des erreurs de gestion de stock.

L'objectif du projet était de mettre en place une plateforme numérique modulaire reposant sur une architecture microservices permettant une gestion distribuée des processus métier, une interopérabilité entre services, et une forte scalabilité.

2. Architecture globale

L'architecture repose sur 9 microservices indépendants, un API Gateway (Kong), un message broker (RabbitMQ) et un frontend Next.js. Voici la décomposition complète :

Couche	Composant	Port	Rôle
Frontend	Next.js App	3000	Interface utilisateur — Login, Dashboard, Gestion
Gateway	Kong Gateway	8000/8001	Point d'entrée unique — routage, sécurité, logs
Auth	Auth Service	3001	JWT, Login, Register, Verify token
Users	User Service	3002	CRUD utilisateurs, gestion rôles Admin/Opérateur/Client
Produits	Product Service	3003	Catalogue produits SFMC
Stocks	Inventory Service	3004	Stock multi-entrepôts, mouvements IN/OUT, seuil critique
Commandes	Order Service	3005	Cycle de vie : pending → validated → shipped → delivered
Production	Production Service	3006	Planification et suivi des lots de fabrication
Facturation	Billing Service	3007	Génération factures, enregistrement paiements
Alertes	Notification Service	3008	Consomme 3 queues RabbitMQ, enregistre les alertes
Analytics	Reporting Service	3009	Agrégation données, dashboard complet
Messaging	RabbitMQ	5672/15672	Message broker asynchrone — exchange sfmc.events

3. Choix technologiques

● Next.js

Framework full stack maîtrisé par toute l'équipe. Les Route Handlers (app/api/) permettent de créer des APIs REST complètes pour chaque micro service, garantissant une cohérence totale.

● Kong Gateway

API Gateway universel Node.js-compatible. Remplace Eureka (Java uniquement) en assurant le routage, la sécurité JWT, les logs et le rate limiting en un seul outil.

● RabbitMQ

Message broker simple à déployer via Docker. Suffisant pour notre scope de TP avec son modèle Queue/Exchange topic, contrairement à Kafka (streaming haute volumétrie + ZooKeeper).

● Prisma 7

ORM TypeScript avec génération automatique des types. Le driver adapter PostgreSQL améliore les performances et la gestion des connexions.

● Docker Compose

Lance Kong + RabbitMQ en une seule commande. Élimine toute configuration manuelle et garantit un environnement identique sur toutes les machines de l'équipe.

---

4. Authentification — Double système

4.1 Credentials (email + mot de passe)

Flux de connexion classique :

- L'utilisateur entre email + mot de passe sur /front/auth/login
- Next Auth appelle CredentialsProvider et cherche l'utilisateur en base via Prisma
- Le mot de passe est comparé avec bcrypt.compare() — jamais stocké en clair
- Si OK → JWT généré dans un cookie httpOnly sécurisé
- Le middleware.ts protège /dashboard et redirige si pas de session

4.2 Google OAuth 2.0

Flux de connexion Google :

- L'utilisateur clique 'Connexion avec Google' → signIn('google')
- Redirection vers l'écran de consentement Google OAuth
- Callback sur /api/auth/callback/google avec le profil Google
- Si l'email n'existe pas en base → création automatique du compte (provider='google')
- Si l'email existe déjà → connexion directe
- JWT créé → cookie session → redirection /dashboard

---

5. RabbitMQ — Pipeline événementiel

Toute la communication asynchrone entre microservices repose sur un exchange RabbitMQ de type topic nommé sfmc.events. Chaque service publie ou consomme des événements selon son rôle.

Les 4 événements du pipeline

Événement	Publisher	Consumer(s)	Action déclenchée
order.created	Order Service	Inventory + Notification	Stock décrémenté · Notification enregistrée
stock.alert	Inventory Service	Production + Notification	Lot planifié auto · Alerte gestionnaire
stock.updated	Production Service	Inventory Service	Stock incrémenté automatiquement
payment.confirmed	Billing Service	Order + Notification	Commande → 'shipped' · Notification client

Flux complet d'une commande — de A à Z

Voici exactement ce qui se passe quand un client commande 50 sacs de ciment :

- 1. Client → POST /api/orders → Order Service crée la commande (statut: pending)
- 2. Order Service public 'order.created' → { orderId, productId, quantity: 50 }
- 3. Inventory reçoit → stock Ciment passe de 500 à 450 unités automatiquement
- 4. Notification reçoit → enregistre 'Nouvelle commande #X : 50x Ciment Portland'
- 5. Si stock < seuil → Inventory publié 'stock.alert'
- 6. Production reçoit → crée automatiquement un lot de 100 sacs (quantity × 2)
- 7. Admin complète le lot → PUT /api/production/1 { status: 'completed' }
- 8. Production publie 'stock.updated' → Inventory incrémente le stock de 100
- 9. Client paie → POST /api/payments → Billing enregistre le paiement
- 10. Billing publie 'payment.confirmed' → Order passe à 'shipped'
- 11. Notification enregistre 'Paiement confirmé, commande expédiée'

6. Kong Gateway

Kong est le point d'entrée unique de notre architecture. Le frontend n'appelle jamais directement les microservices — tout passe par Kong sur le port 8000, qui route vers le bon service, applique la sécurité JWT et les logs.

Conteneurs Docker Compose

Conteneur	Image	Port(s)	Rôle
kong-database	postgres:15	5432 (interne)	Base PostgreSQL dédiée à Kong
kong-migration	kong:3.6	—	Initialise le schéma Kong (une seule fois)
kong	kong:3.6	8000 / 8001	Proxy (8000) · Admin API (8001)
rabbitmq	rabbitmq:3-management	5672 / 15672	AMQP (5672) · Dashboard Web (15672)

7. Database per Service

Chaque microservice dispose de sa propre base de données totalement indépendante. Ce pattern fondamental garantit l'isolation des données, la liberté de schéma et la scalabilité individuelle de chaque service.

Service	Base de données	Tables principales
Frontend (principal)	apiProjet	users, password_reset_tokens
Auth Service	auth_db	auth_users
User Service	user_db	users
Product Service	product_db	products
Inventory Service	inventory_db	stocks, movements
Order Service	order_db	orders
Production Service	production_db	production_batches
Billing Service	billing_db	invoices, payments
Notification Service	notification_db	notifications
Reporting Service	reporting_db	reports

Avantages du pattern

- Indépendance totale : si la base de Order Service tombe, Product Service continue
- Liberté de schéma : chaque service peut avoir le modèle adapté à ses besoins
- Scalabilité : migration possible vers un serveur plus puissant sans impacter les autres
- Sécurité : une faille dans un service n'expose pas les données des autres

---

8. Interfaces de l'application

Cette section présente les principales interfaces du portail SFMC Bénin. Chaque écran correspond à un ou plusieurs microservices actifs en arrière-plan.

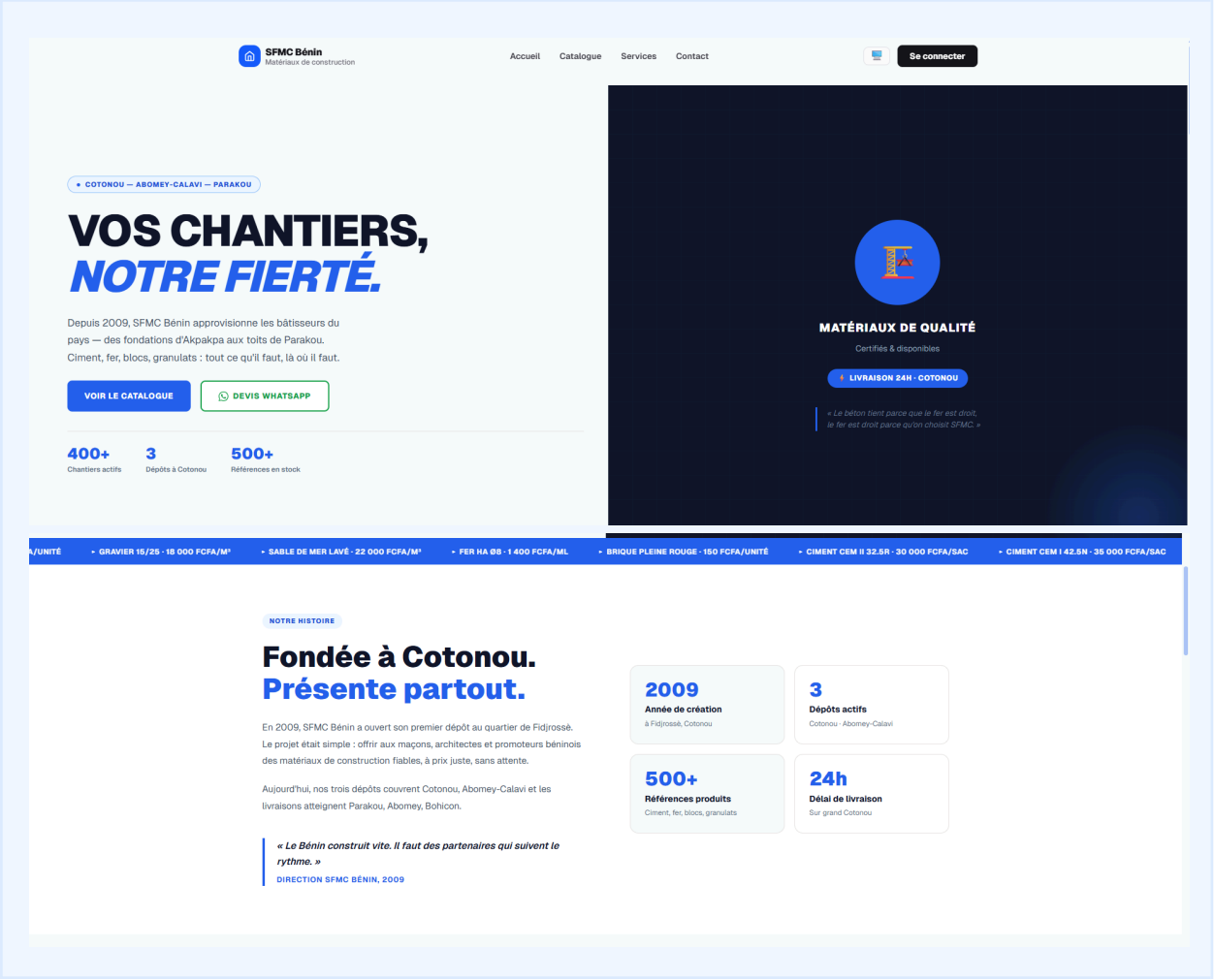
8.1 Page d'accueil publique

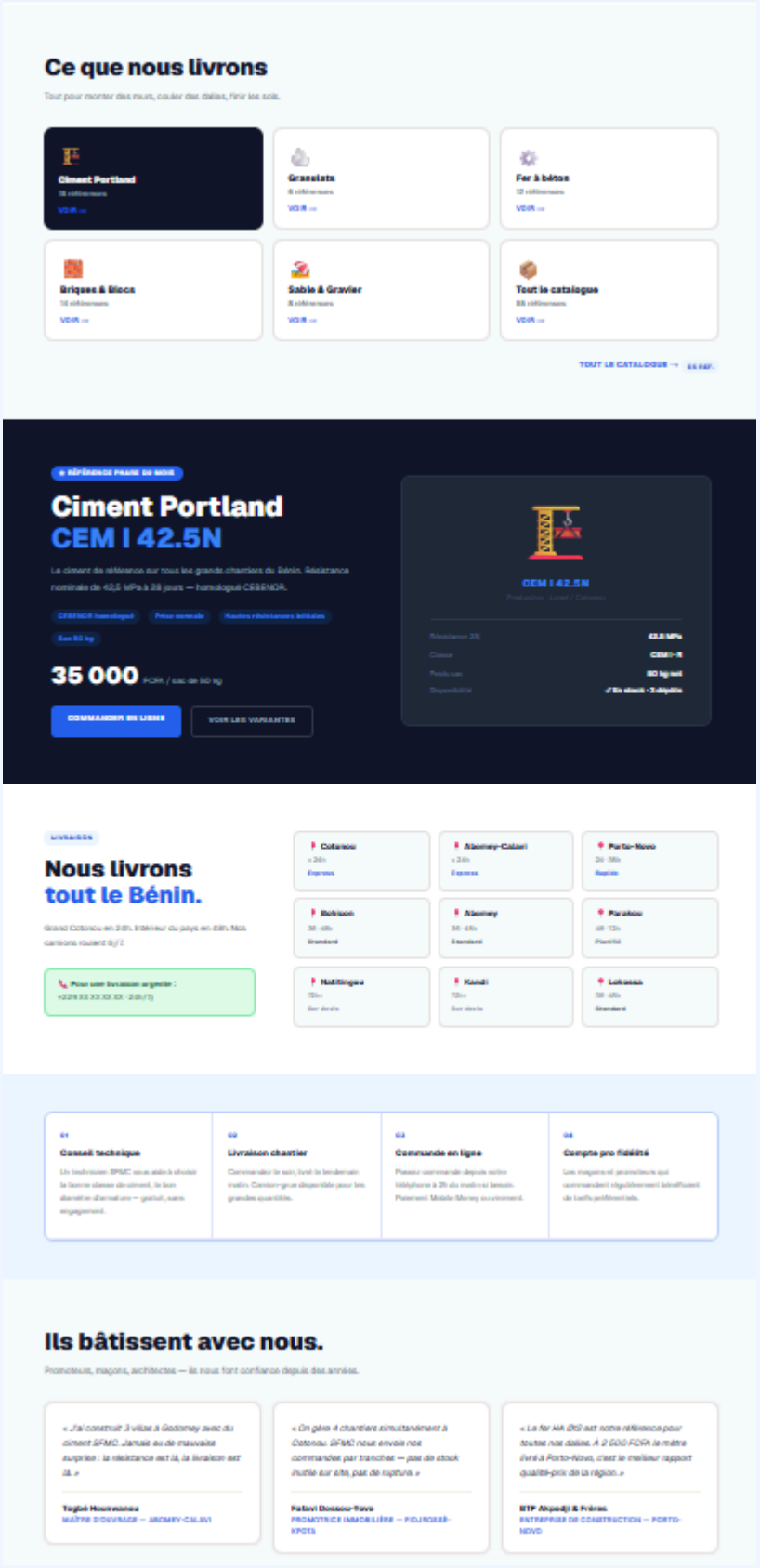
Landing page de la plateforme SFMC Bénin accessible sans authentification. Elle présente l'entreprise, le catalogue produits via une grille bento, les zones de livraison, les témoignages clients et un appel à l'action pour créer un compte. Entièrement responsive (mobile, tablette, desktop).

Fonctionnalités visibles sur cet écran :

- Hero en 2 colonnes avec bande défilante de prix en temps réel
- Grille bento des catégories de produits (ciment, fer, briques, granulats)
- Section zones desservies avec délais de livraison par ville
- Témoignages clients et CTA final

Page d'accueil publique


SFMC Bénin
Matériaux de construction
Accueil
Catalogue
Services
Contact
Se connecter
COTONOU — ABOMEY-CALAVI — PARAKOU
VOS CHANTIERS,
NOTRE FIERTÉ.
Depuis 2009, SFMC Bénin approvisionne les bâtisseurs du pays — des fondations d'Akpakpa aux toits de Parakou. Ciment, fer, blocs, granulats : tout ce qu'il faut, là où il faut.
VOIR LE CATALOGUE
DEVIS WHATSAPP
400+
Chantiers actifs
3
Dépôts à Cotonou
500+
Références en stock
MATÉRIAUX DE QUALITÉ
Certifiés & disponibles
LIVRAISON 24H · COTONOU
« Le béton tient parce que le fer est droit, le fer est droit parce qu'on choisit SFMC. »
GRAVIER 15/25 · 18 000 FCFA/M³
SABLE DE MER LAVÉ · 22 000 FCFA/M³
FER HA Ø8 · 1 400 FCFA/ML
BRIQUE PLEINE ROUGE · 150 FCFA/UNITÉ
CIMENT CEM II 32.5R · 30 000 FCFA/SAC
CIMENT CEM I 42.5N · 35 000 FCFA/SAC
NOTRE HISTOIRE
Fondée à Cotonou.
Présente partout.
En 2009, SFMC Bénin a ouvert son premier dépôt au quartier de Fidjrossè. Le projet était simple : offrir aux maçons, architectes et promoteurs béninois des matériaux de construction fiables, à prix juste, sans attente.
Aujourd'hui, nos trois dépôts couvrent Cotonou, Abomey-Calavi et les livraisons atteignent Parakou, Abomey, Bohicon.
« Le Bénin construit vite. Il faut des partenaires qui suivent le rythme. »
DIRECTION SFMC BÉNIN, 2009
2009
Année de création
à Fidjrossè, Cotonou
3
Dépôts actifs
Cotonou · Abomey-Calavi
500+
Références produits
Ciment, fer, blocs, granulats
24h
Délai de livraison
Sur grand Cotonou


Ce que nous livrons
Ciment Portland
Granulats
Fer à béton
Briques & Blocs
Sable & Gravier
Tout le catalogue
TOUT LE CATALOGUE →
Ciment Portland
CEM I 42.5N
35 000
COMMANDER EN LIGNE
VOIR LES VARIANTES
CEM I 42.5N
Nous livrons
tout le Bénin.
Cotonou
Abomey-Calavi
Porto-Novo
Bohicon
Abomey
Parakou
Natitingou
Kandi
Lokossa
Conseil technique
Livraison chantier
Commande en ligne
Compte pro fidélité
Ils bâtissent avec nous.
« J'ai construit 3 villas à Godomey avec du ciment SFMC. Jamais eu de mauvaise surprise : la résistance est là, la livraison est là. »
Tegbé Hounmanou
MAÎTRE D'OUVRAGE — ABOMEY-CALAVI
« On gère 4 chantiers simultanément à Cotonou. SFMC nous envoie nos commandes par tranches — pas de stock inutile sur site, pas de rupture. »
Fatovi Dossou-Yovo
PROMOTRICE IMMOBILIÈRE — FIDJROSSÈ-KPOTA
« Le fer HA Ø10 est notre référence pour toutes nos dalles. À 2 500 FCFA le mètre livré à Porto-Novo, c'est le meilleur rapport qualité-prix de la région. »
BTP Akpadji & Frères
ENTREPRISE DE CONSTRUCTION — PORTO-NOVO

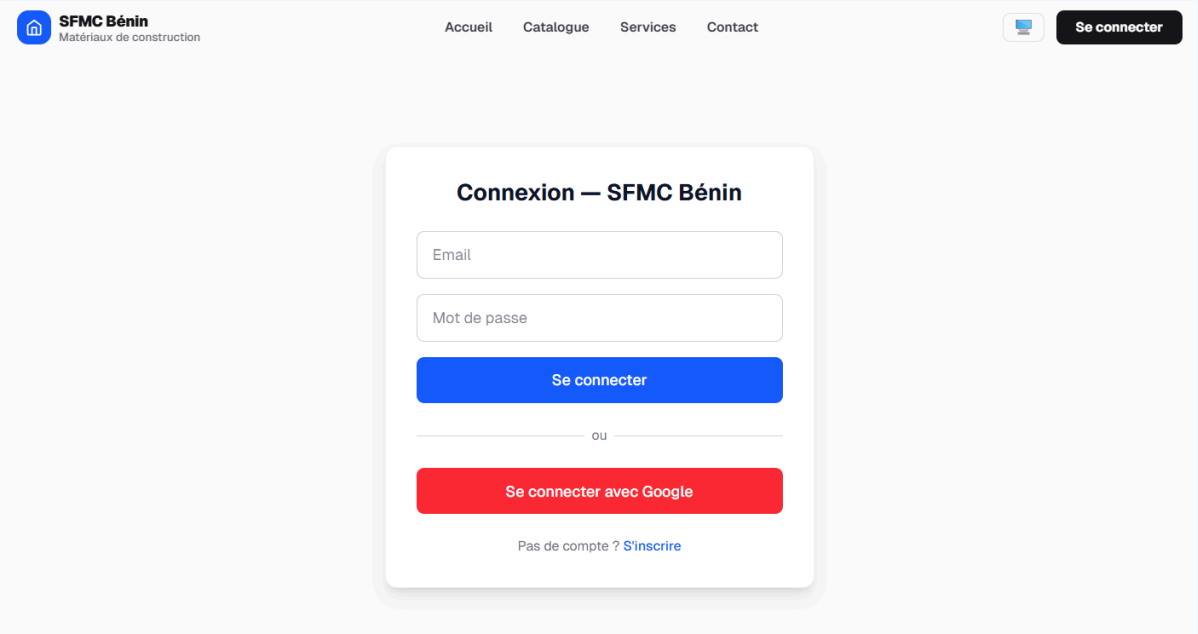
8.2 Authentification — Connexion et Inscription

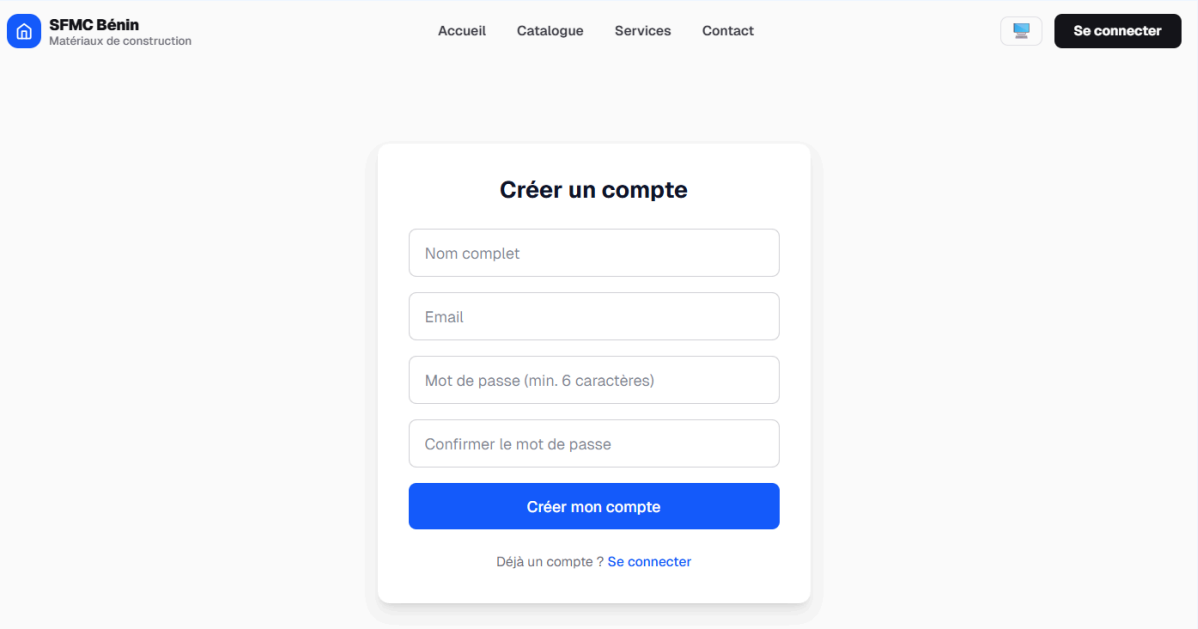
Pages d'authentification avec double système : connexion par email/mot de passe (JWT + bcrypt) et OAuth Google 2.0. Le middleware Next.js protège toutes les routes /dashboard et redirige automatiquement si la session est absente.

Fonctionnalités visibles sur cet écran :

- Formulaire login avec validation côté client et serveur
- Connexion Google OAuth 2.0 en un clic
- Création de compte avec hash bcrypt automatique
- Gestion des erreurs (email déjà utilisé, mot de passe incorrect)

Authentification — Connexion et Inscription





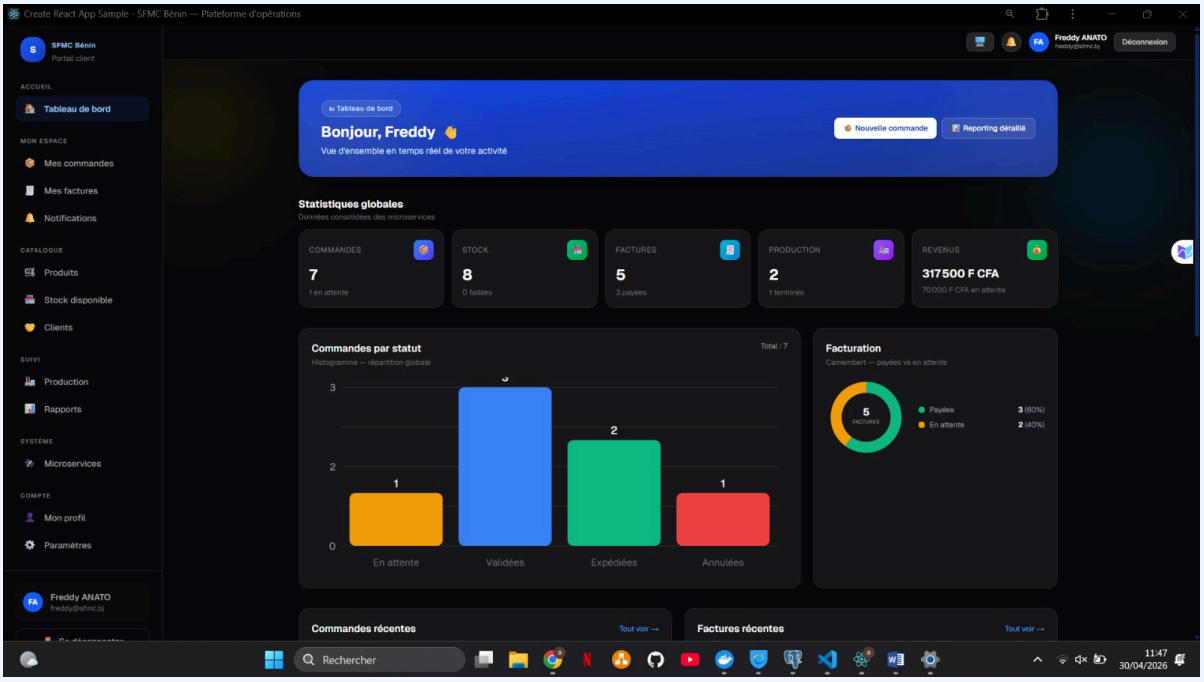
8.3 Tableau de bord analytics

Dashboard principal accessible après authentification. Affiche des KPIs consolidés en temps réel depuis le Reporting Service, un histogramme des commandes par statut et un camembert des factures — tous en SVG natif sans dépendance externe.

Fonctionnalités visibles sur cet écran :

- 5 cartes KPI : Commandes, Stock, Factures, Production, Revenus FCFA
- Histogramme SVG natif des commandes par statut
- Camembert SVG des factures (payées vs en attente)
- Widget live : statut Kong, RabbitMQ et microservices

Tableau de bord analytics



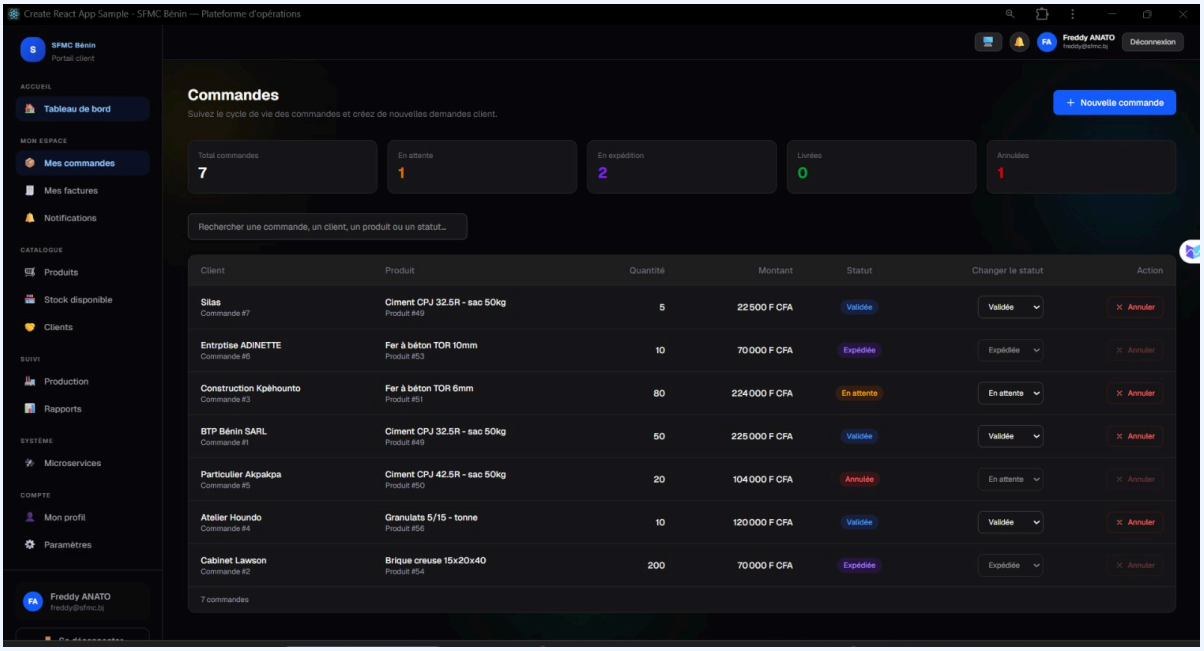
8.4 Gestion des commandes

Interface de suivi du cycle de vie des commandes. Chaque commande passe par les états pending → validated → shipped → delivered, orchestrés par l'Order Service en coordination avec l'Inventory Service via RabbitMQ.

Fonctionnalités visibles sur cet écran :

- Liste paginée des commandes avec filtres par statut
- Création de commande avec vérification automatique du stock
- Mise à jour de statut en temps réel
- Déclenchement automatique de production si stock insuffisant

Gestion des commandes



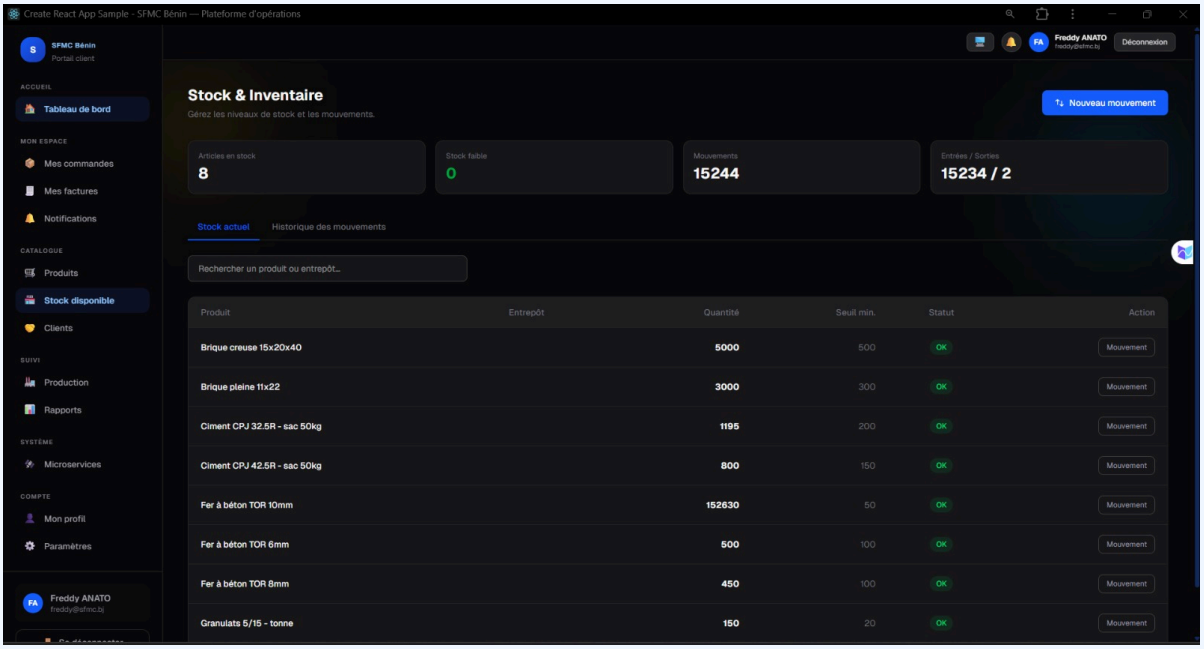
8.5 Gestion des stocks (Inventory)

Interface multi-entrepôts pour suivre les niveaux de stock de chaque produit. L'Inventory Service consomme les événements RabbitMQ pour mettre à jour les stocks automatiquement et déclencher des alertes si le seuil critique est atteint.

Fonctionnalités visibles sur cet écran :

- Vue multi-entrepôts avec niveaux de stock en temps réel
- Historique des mouvements IN/OUT avec raison et date
- Alertes visuelles sur les produits en dessous du seuil critique
- Synchronisation automatique via événements order.created et stock.updated

Gestion des stocks (Inventory)



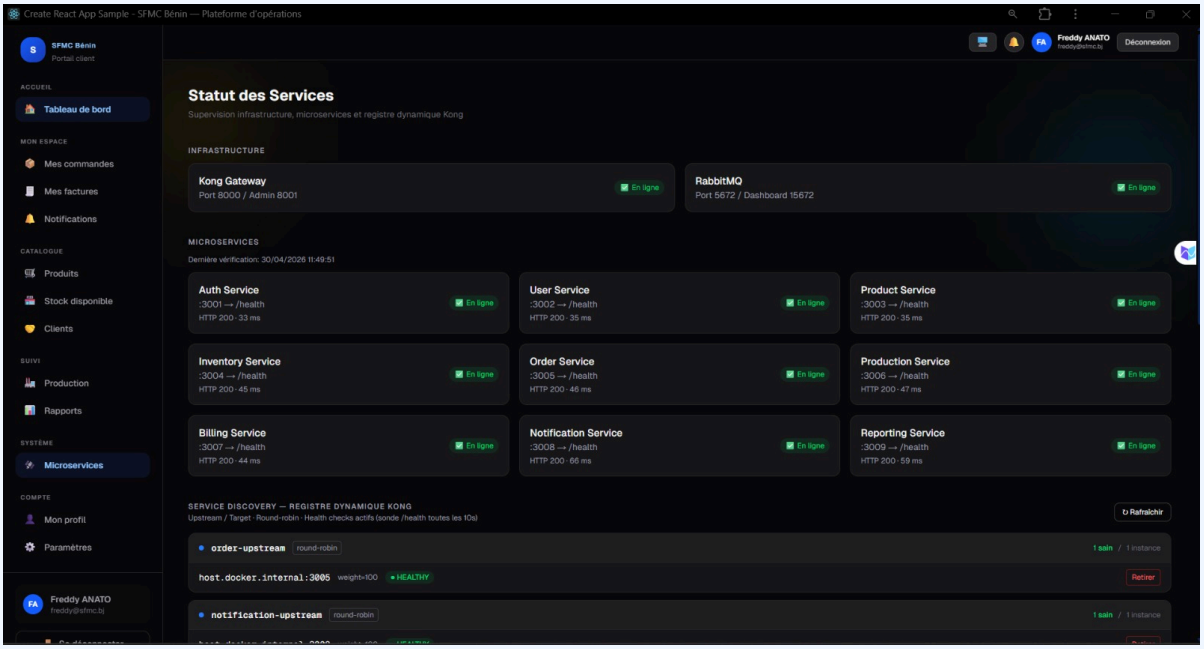
8.6 Microservices — Supervision de l'infrastructure

Page de supervision dédiée à l'état de l'infrastructure. Affiche en temps réel le statut de Kong Gateway, RabbitMQ et de chacun des 9 microservices, avec pastilles d'état colorées (vert/rouge) et horodatage de la dernière vérification.

Fonctionnalités visibles sur cet écran :

- Pastilles live : Kong, RabbitMQ et 9 microservices
- Statut HTTP de chaque service (200 OK / 503)
- Bouton Actualiser pour rafraîchir les statuts manuellement
- Accessible depuis le menu Système → Microservices

Microservices — Supervision de l'infrastructure



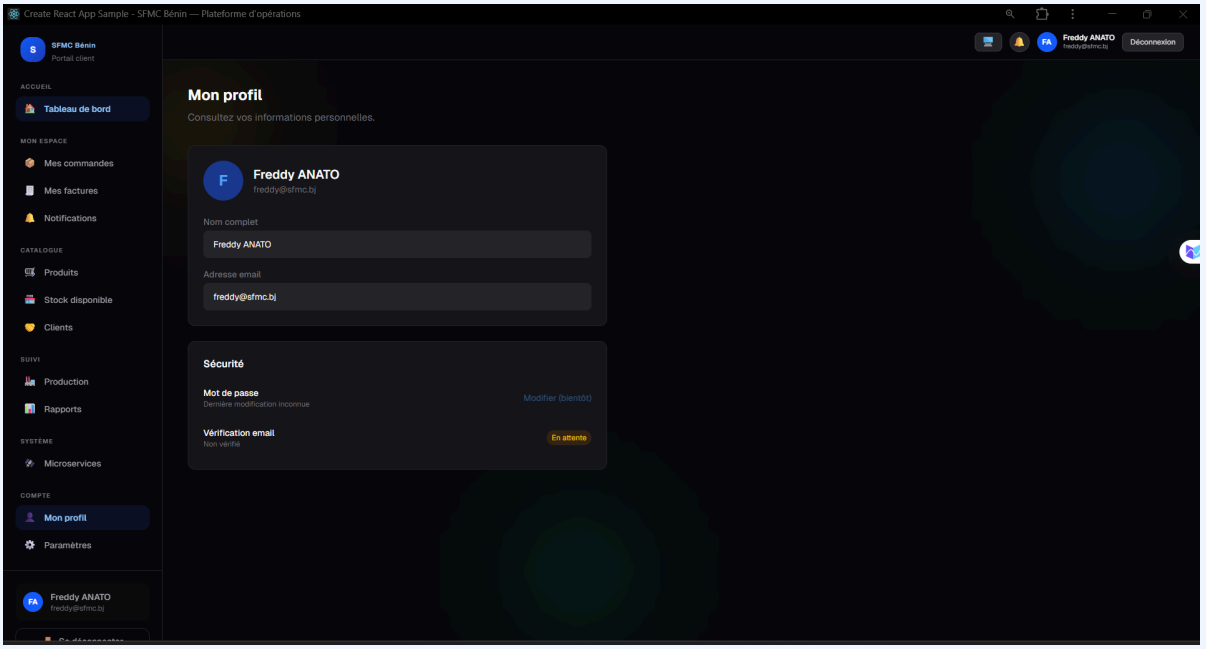
8.7 Thème utilisateur et Paramètres

Page de paramètres permettant à chaque utilisateur de choisir son thème d'affichage (Clair, Sombre, Système) avec persistance via localStorage et application instantanée sans rechargement. Un script anti-FOUC garantit l'absence de flash blanc au démarrage.

Fonctionnalités visibles sur cet écran :

- 3 modes : Clair, Sombre, Système (suit le thème OS)
- Persistance localStorage — le choix survit aux rechargements
- Script anti-FOUC inline dans <head> pour éviter le flash blanc
- Toggle accessible depuis la navbar (desktop + mobile)

Thème utilisateur et Paramètres



9. Structure d'un microservice

Exemple avec l'Inventory Service (port 3004). Tous les microservices partagent la même structure Next.js.

Fichier	Rôle
app/api/stock/route.ts	GET tous les stocks + POST créer un stock
app/api/movements/route.ts	GET mouvements + POST enregistrer IN/OUT + mise à jour stock
app/api/init/route.ts	GET → démarre le consumer RabbitMQ (order.created)
lib/prisma.ts	Client Prisma avec driver adapter pg + pattern singleton
lib/consumer.ts	Écoute 'order.created' → décrémente stock automatiquement
lib/rabbitmq.ts	Publisher → publie 'stock.alert' quand stock < seuil
prisma/schema.prisma	Modèles Stock et Movement — schéma de la base inventory_db
prisma.config.ts	Configuration Prisma 7 avec defineConfig + datasource url
.env	DATABASE_URL + RABBITMQ_URL (jamais commité sur Git)
package.json	scripts: dev sur port 3004, build, start